CSA – 0610

DESIGN AND ANALYSIS OF ALGORITHMS

SLOT – A

ASSIGNMENT - 1

DONE BY: DIVYADHARSHINI V S

ROLL NO: 192511308

ASSIGNMENT – 1

TASK 1 – DESIGN AN ALGORITHM

1.1 Introduction

The objective is to find a student's registration number from a collection of student records. Three algorithms are considered: Linear Search, Recursive Binary Search, and Hash-Based Search. Imagine finding a student's name in a college register. Linear Search checks the entries one by one. Binary Search repeatedly divides a sorted list into halves. Hash-Based Search calculates a location where the required record should be stored.

1.2 Algorithm 1: Linear Search

Linear Search checks each element from the beginning until the target is found or the list ends. It works with both sorted and unsorted data.

Algorithm:

1. Start from the first element.
2. Compare the current element with the target registration number.
3. If both are equal, return the position.
4. Otherwise, move to the next element.
5. If all elements have been checked, return NOT FOUND.

Pseudocode:

```
LINEAR-SEARCH(A, n, K)

FOR i = 0 TO n - 1

   IF A[i] = K

      RETURN i

RETURN NOT FOUND
```

Here, A is the array, n is the number of records, and K is the target registration number.

Example:

Consider the registration numbers:

101, 205, 309, 412, 518

If the target is 412, the algorithm compares it with 101, 205, 309 and finally 412. The target is found at index 3.

Time Complexity: O(n), because the algorithm may check every element.

1.3 Algorithm 2: Recursive Binary Search

Binary Search works only when the elements are sorted. It checks the middle element and continues searching in only one half of the list.

Algorithm:

1. Find the middle position.
2. If the middle element equals the target, return its position.
3. If the target is smaller, search the left half.
4. Otherwise, search the right half.
5. Repeat until the target is found or the search range becomes empty.

Pseudocode:

```
BINARY-SEARCH(A, low, high, K)
IF low > high
   RETURN NOT FOUND
mid = low + (high - low) / 2
IF A[mid] = K
   RETURN mid
IF K < A[mid]
   RETURN BINARY-SEARCH(A, low, mid - 1, K)
RETURN BINARY-SEARCH(A, mid + 1, high, K)
```

Example:

For 1,024 sorted records, Binary Search checks the middle and eliminates half of the remaining records at each step. Approximately 10 halving steps are sufficient because $2^{10} = 1,024$.

Time Complexity: O(log n), because the search range is repeatedly divided by two.

If the records are not already sorted, sorting them using an efficient comparison-based sorting algorithm takes O(n log n) time.

1.4 Algorithm 3: Hash-Based Search

Hash-Based Search uses a hash function to calculate the location of a registration number in a hash table.

Algorithm:

1. Create a hash table.
2. Insert each registration number into its calculated location.
3. Calculate the location of the target registration number.
4. Check the corresponding slot or bucket.
5. Return the record if found; otherwise, return NOT FOUND.

Pseudocode:

```
HASH-SEARCH(TABLE, K)
bucket = HASH(K)
FOR each record in TABLE[bucket]
   IF record.registration = K
      RETURN record
RETURN NOT FOUND
```

Example:

Suppose the hash function is:

HASH(K) = K MOD 101

For registration number 412:

412 MOD 101 = 9

Therefore, the search begins at hash-table slot 9.

If multiple registration numbers map to the same location, a collision occurs. A collision-handling technique such as separate chaining or open addressing must be used.

Time Complexity: Expected O(1) for a well-designed hash table, but O(n) in the worst case.

1.5 Comparison and Conclusion

Algorithm	Main Approach	Requirement	Worst-Case Search
Linear Search	Checks elements one by one	No sorting required	O(n)
Binary Search	Repeatedly halves the search range	Sorted data	O(log n)
Hash-Based Search	Calculates the likely location	Hash table	O(n)
Hash-Based Search — Expected Case	Checks a small number of entries	Suitable hashing and table size	O(1)

Linear Search is simple and works with unsorted records. Binary Search is efficient when the records are sorted. Hash-Based Search is suitable for frequent exact registration-number lookups because it provides expected constant-time searching.

TASK 2 – IDENTIFY THE BASIC OPERATION

2.1 Meaning of Basic Operation

A basic operation is the main repeated action performed by an algorithm. Identifying it helps estimate the amount of work required as the input size increases.

For student-record searching, the main operation is usually comparing the target registration number with a stored registration number.

2.2 Linear Search

The basic operation is:

A[i] == K

The algorithm compares each element with the target.

Consider:

101, 205, 309, 412, 518

If the target is 101, only one comparison is needed. If the target is 518, five comparisons are needed. If the target is absent, all five elements must be checked.

For n records, the worst case requires n comparisons.

Therefore, the worst-case time complexity is O(n).

2.3 Binary Search

The basic operation is:

A[mid] == K

The algorithm also checks whether the target is smaller than the middle element.

Each comparison eliminates approximately half of the remaining records.

For example, searching 1,024 sorted records requires approximately 10 halving steps rather than checking all 1,024 records individually.

Therefore, Binary Search has O(log n) worst-case time complexity.

2.4 Hash-Based Search

The basic operation is comparing the target registration number with a key stored in the calculated hash-table location.

The algorithm first calculates the hash index and then checks the corresponding slot or bucket.

If the bucket contains very few records, only a small number of comparisons are needed. Collisions can increase the number of comparisons.

Therefore, Hash-Based Search has expected O(1) lookup time and O(n) worst-case lookup time.

2.5 Comparison Table

Algorithm	Basic Operation	Worst-Case Work
Linear Search	Compare A[i] with K	n comparisons
Binary Search	Compare A[mid] with K	O(log n) comparisons
Hash-Based Search	Compare keys in the selected bucket or probe sequence	O(n) comparisons
Hash-Based Search — Expected Case	Compare keys in a short bucket or probe sequence	O(1) expected comparisons

2.6 Conclusion

Counting basic operations provides a fair way to compare algorithms independently of computer speed. Linear Search examines records one by one, Binary Search repeatedly halves the search range, and Hash-Based Search normally examines only a small number of entries.

TASK 3 – DERIVE TIME COMPLEXITY MATHEMATICALLY

3.1 Linear Search

Let n represent the number of student records.

In the worst case, the target is the last element or is absent. Therefore, all n elements must be checked.

The number of comparisons is:

$C(n) = n$

If each comparison takes constant time c and the remaining overhead is d, the running time can be represented as:

$T(n) = cn + d$

For asymptotic analysis, constant factors and lower-order terms are ignored.

Therefore:

$T(n) = \Theta(n)$

Hence, the worst-case time complexity is **O(n)**.

Why O(n)?

If the number of records doubles, the maximum number of comparisons also approximately doubles.

For example:

- 100 records may require 100 comparisons.
- 200 records may require 200 comparisons.
- 1,000 records may require 1,000 comparisons.

The work increases proportionally with the input size.

3.2 Binary Search

Binary Search divides the search interval into two halves at each step.

After one step, the remaining size is approximately n/2.

After two steps, it is approximately n/4.

After k steps, it is approximately:

$n / 2^k$

The search reaches one element when:

$n / 2^k \leq 1$

Therefore:

$2^k \geq n$

Taking logarithms:

$k \geq \log_2 n$

Hence, the number of steps grows logarithmically.

Therefore:

$T(n) = \Theta(\log n)$

The worst-case time complexity is **O(log n)**.

For 1,024 records:

$\log_2 1{,}024 = 10$

Therefore, approximately 10 halving steps are sufficient.

3.3 Sorting Cost for Binary Search

Binary Search requires sorted data.

If the records are initially unsorted, sorting them using an efficient comparison-based sorting algorithm takes O(n log n) time.

If q searches are performed, the total cost is:

$T(n, q) = O(n \log n + q \log n)$

If the records are already sorted, q searches require:

$T(q) = O(q \log n)$

Therefore, preprocessing must be considered when comparing algorithms.

3.4 Hash-Based Search

Building a hash table for n records takes expected O(n) time when the hash function distributes keys appropriately.

An individual search takes expected O(1) time.

For q searches, the expected total cost is:

$T(n, q) = O(n + q)$

However, if many registration numbers collide in the same bucket, a search may take O(n) time in the worst case.

3.5 Mathematical Comparison Table

Algorithm	Best-Case Time	Average/Expected Time	Worst-Case Time
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Hash-Based Search	O(1)	Expected O(1)	O(n)

3.6 Conclusion

Linear Search grows linearly, Binary Search grows logarithmically, and Hash-Based Search normally provides constant-time lookup under suitable hashing conditions. Preprocessing costs must also be included when comparing the complete systems.

TASK 4 – BEST, AVERAGE AND WORST CASE

4.1 Linear Search

Best Case

The target is located at the first position. Only one comparison is required.

Time Complexity: Θ(1)

Average Case

Assuming each position is equally likely for a successful search, the average number of comparisons is:

Average comparisons = (n + 1) / 2

Therefore, the average time complexity is Θ(n).

Worst Case

The target is at the last position or does not exist. The algorithm may check all n elements.

Time Complexity: Θ(n)

Real-life comparison: Finding the first name in a printed register is quick. Finding a name near the end takes longer. If the name is missing, the entire register may need to be checked.

4.2 Binary Search

Best Case

The target matches the middle element during the first comparison.

Time Complexity: Θ(1)

Average Case

The algorithm repeatedly divides the sorted search interval into halves.

Time Complexity: Θ(log n)

Worst Case

The algorithm continues halving the range until the target is found or the range becomes empty.

Time Complexity: Θ(log n)

4.3 Hash-Based Search

Expected Case

With a suitable hash function, appropriate table size and controlled load factor, the selected bucket contains only a small number of entries.

Expected Time Complexity: Θ(1)

Worst Case

If many registration numbers collide into the same bucket, the algorithm may inspect all n records.

Worst-Case Time Complexity: Θ(n)

Building the table separately requires expected O(n) time.

4.4 Comparison Table

Algorithm	Best Case	Average/Expected Case	Worst Case
Linear Search	Θ(1)	Θ(n)	Θ(n)
Binary Search	Θ(1)	Θ(log n)	Θ(log n)
Hash-Based Search	Θ(1)	Θ(1), expected	Θ(n)

4.5 Conclusion

Linear Search is suitable for small or unsorted collections. Binary Search is effective when records are sorted. Hash-Based Search is generally preferable for frequent exact registration-number searches, provided collisions are controlled.

TASK 5 – ASYMPTOTIC ANALYSIS: O, Ω AND Θ

5.1 Big-O Notation

Big-O notation describes an asymptotic upper bound on an algorithm's growth.

It indicates that, beyond a sufficiently large input size, the running time does not grow faster than a constant multiple of the stated function.

For Linear Search, the worst-case number of comparisons is n.

Therefore:

$T(n) = O(n)$

For Binary Search, the search interval is repeatedly halved.

Therefore:

$T(n) = O(\log n)$

For Hash-Based Search, expected lookup time is O(1) under suitable hashing assumptions, while worst-case time is O(n).

Important: Big-O is an upper-bound notation. It is commonly used to report worst-case complexity, but Big-O does not automatically mean worst case.

5.2 Omega Notation

Omega notation describes an asymptotic lower bound.

It indicates that the running time grows at least as fast as the stated bound for the case being analysed.

For a non-empty array, a successful search normally needs at least one comparison.

Therefore, the best-case lower bound for Linear Search is:

$\Omega(1)$

Binary Search also has a best-case lower bound of:

$\Omega(1)$

The best case occurs when the target is found immediately.

5.3 Theta Notation

Theta notation describes a tight asymptotic bound. It means the function has both an upper and a lower bound of the same growth order.

Examples:

- Linear Search worst case: Θ(n)
- Binary Search worst case: Θ(log n)
- Hash-Based Search expected case: Θ(1), under suitable hashing assumptions
- Hash-Based Search worst case: Θ(n)

5.4 Comparison Table

Notation	Meaning	Example
O(g(n))	Upper bound	Linear Search worst case: O(n)
Ω(g(n))	Lower bound	Immediate successful search: Ω(1)
Θ(g(n))	Tight bound	Binary Search worst case: Θ(log n)

5.5 Conclusion

O, Ω and Θ describe mathematical bounds rather than different algorithms. The selected bound must match the case being analysed. Assumptions such as sorted input, hash distribution and preprocessing must be stated clearly.

TASK 6 – FORMULATE A RECURRENCE RELATION

6.1 Introduction

A recurrence relation expresses the running time of a recursive algorithm in terms of the running time of a smaller problem.

Recursive Binary Search is suitable for recurrence analysis because each call reduces the search interval.

6.2 Step 1: Identify the Input Size

Let n be the number of records remaining in the current search interval.

Initially, n may represent the entire collection.

After comparing the middle element with the target, the algorithm continues searching in only one half.

Therefore, the next problem size is approximately n/2.

6.3 Step 2: Identify the Constant Work

Each recursive call performs a few operations:

- Calculates the middle index.
- Compares the middle element with the target.
- Determines which half should be searched.
- Makes one recursive call or returns a result.

These operations require constant time, represented by c.

6.4 Step 3: Form the Recurrence

The algorithm makes only one recursive call on a problem of size approximately n/2.

Therefore, the recurrence is:

$T(n) = T(n/2) + c$, for $n > 1$

The base case is:

$T(1) = c_0$

Here, c and c_0 are constants.

6.5 Why Is It Not 2T(n/2)?

A recurrence such as:

$T(n) = 2T(n/2) + c$

represents an algorithm that recursively solves both halves.

Binary Search does not search both halves. It selects only the half that may contain the target.

Therefore, the correct recurrence contains only one recursive term.

6.6 Real-Life Comparison

Imagine finding a numbered locker among 1,024 lockers arranged in sorted order. You check the middle locker and eliminate half of the remaining lockers. You repeat this process until only one or zero possible lockers remain. The number of lockers decreases rapidly, but the algorithm makes only one recursive call at each stage.

6.7 Conclusion

The recurrence for Recursive Binary Search is:

$T(n) = T(n/2) + c$

This represents one recursive subproblem of half the size plus constant extra work. Solving the recurrence gives a time complexity of $\Theta(\log n)$.

TASK 7 – SOLVE THE RECURRENCE RELATION

7.1 Given Recurrence

The recurrence relation for Recursive Binary Search is:

$T(n) = T(n/2) + c$

Base case:

$T(1) = c_0$

Here, c represents the constant work performed during each call. For easier calculation, assume n is a power of two. The final asymptotic result remains the same for other input sizes.

7.2 Method 1: Iteration Method

Start with:

$T(n) = T(n/2) + c$

Expand T(n/2):

$T(n/2) = T(n/4) + c$

Substitute:

$T(n) = T(n/4) + 2c$

Expand again:

$T(n) = T(n/8) + 3c$

After k expansions:

$T(n) = T(n/2^k) + kc$

The recursion stops when the problem size becomes one.

Therefore:

$n/2^k = 1$

Rearranging:

$2^k = n$

Taking logarithms:

$k = \log_2 n$

Substituting this value:

$T(n) = T(1) + c \log_2 n$

Since T(1) is a constant:

$T(n) = \Theta(\log n)$

Therefore, Recursive Binary Search has logarithmic time complexity.

7.3 Method 2: Substitution Method

From the recurrence:

$T(n) = T(n/2) + c$

Assume that the running time is proportional to log n.

Then:

$T(n/2) = \Theta(\log_2(n/2))$

Using the logarithm identity:

$\log_2(n/2) = \log_2 n - 1$

Substituting:

$T(n) = \Theta(\log_2 n - 1) + c$

The subtraction of a constant and addition of constant work do not change the growth order.

Therefore:

$T(n) = \Theta(\log n)$

The substitution method confirms the result obtained through iteration.

7.4 Method 3: Master Theorem

The general recurrence is:

$T(n) = aT(n/b) + f(n)$

For Binary Search:

$a = 1, b = 2$

$f(n) = \Theta(1)$

Calculate:

$n^{(\log_b a)} = n^{(\log_2 1)}$ Since $\log_2 1 = 0$:

$n^0 = 1$

Therefore:

$f(n) = \Theta(n^{(\log_b a)}) = \Theta(1)$

This matches the balanced case of the Master Theorem.

Hence:

$T(n) = \Theta(\log n)$

7.5 Practical Interpretation

For 1,024 records:

$\log_2 1{,}024 = 10$

Binary Search requires approximately ten halving steps.

For around one million records, approximately twenty halving steps are sufficient because:

$2^{20} = 1{,}048{,}576$

This shows why Binary Search can locate a record efficiently in a large sorted collection.

7.6 Conclusion

The iteration method, substitution method and Master Theorem all produce the same result.

Final complexity: T(n) = Θ(log n)

This result applies to the search operation when the data is sorted. The cost of sorting unsorted records must be considered separately.

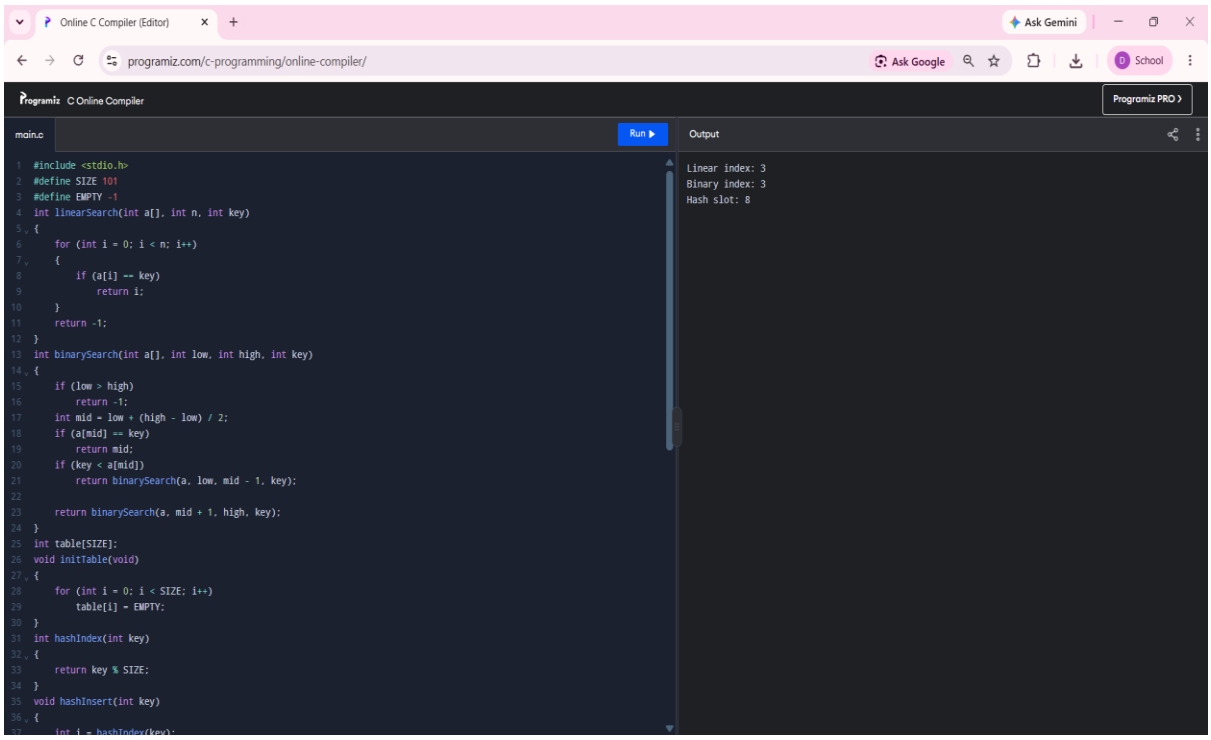
TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

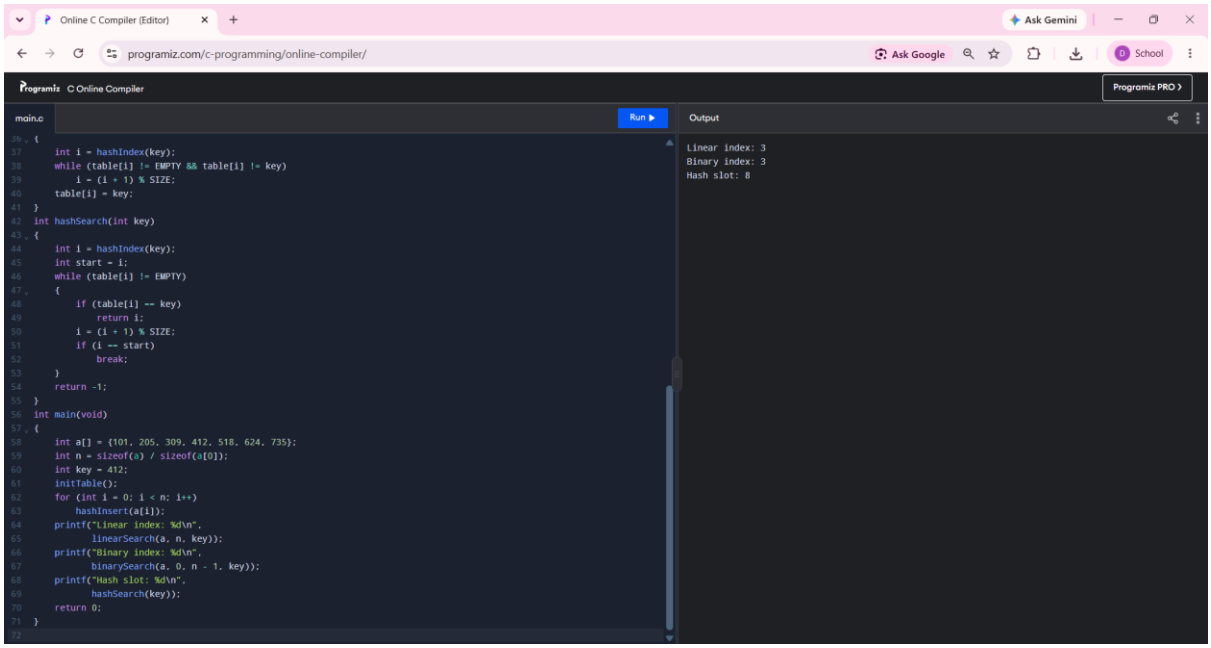
8.1 Aim

The aim is to implement Linear Search, Recursive Binary Search and Hash-Based Search using C. The program searches for a registration number and displays the result of each algorithm.

The sample registration numbers are sorted so that Binary Search can be used correctly.

8.2 C Program





8.3 Explanation of Important Statements

Header File

#include <stdio.h>

This includes the standard input/output library and allows the program to display results using printf().

Linear Search Function

for (int i = 0; i < n; i++)

The loop checks each array element from the first position to the last.

if (a[i] == key)

This checks whether the current registration number matches the target.

return i;

If the target is found, the function returns its array index.

return -1;

If the loop finishes without finding the target, the function returns −1 to indicate that the value is absent.

Binary Search Function

```
if (low > high)
    return -1;
```

This is the base condition. It stops recursion when no elements remain in the search interval.

```
int mid = low + (high - low) / 2;
```

This calculates the middle position.

```
return binarySearch(a, low, mid - 1, key);
```

If the target is smaller than the middle element, the function searches the left half.

```
return binarySearch(a, mid + 1, high, key);
```

Otherwise, it searches the right half.

Only one recursive call is made because the target can be in only one of these halves.

Hash-Based Search

```
return key % SIZE;
```

This calculates the initial hash-table location using the remainder operator.

```
while (table[i] != EMPTY && table[i] != key)
```

If a slot is occupied by a different key, the program moves to the next slot. This is called linear probing.

8.4 Expected Output

Linear index: 3

Binary index: 3

Hash slot: 9

8.5 Output Explanation

The target is 412.

In the original array, 412 is stored at index 3 because array indexing starts from zero.

Therefore, Linear Search and Binary Search both return index 3.

For Hash-Based Search:

412 % 101 = 9

Therefore, the initial hash location is slot 9. No earlier collision occupies that slot in this example.

The hash-table slot is different from the original array index.

8.6 Complexity Table

Algorithm	Worst-Case Search Complexity	Additional Requirement
Linear Search	O(n)	None
Recursive Binary Search	O(log n)	Sorted input
Hash-Based Search	O(n)	Hash table and collision handling
Hash-Based Search — Expected	O(1)	Suitable hash distribution and capacity

The example assumes non-negative registration numbers other than −1 because −1 represents an empty slot.

8.7 Conclusion

The program demonstrates three search approaches. Linear Search examines elements sequentially, Binary Search repeatedly halves a sorted range, and Hash-Based Search calculates a location and handles collisions through linear probing.

TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

9.1 Objective

The objective is to compare the performance of Linear Search, Binary Search and Hash-Based Search for different dataset sizes.

Theoretical analysis explains how the algorithms should scale, while experiments help evaluate their behaviour on a real computer.

9.2 Experimental Setup

The following dataset sizes are considered:

- 1,000 records
- 10,000 records
- 100,000 records
- 1,000,000 records

The same registration numbers and search targets should be used wherever possible.

Binary Search must use sorted data. The hash table must be built before the lookup tests.

9.3 Test Cases

Three types of searches are considered:

1. Successful search near the beginning.
2. Successful search near the end.
3. Unsuccessful search where the target is absent.

Each test should be repeated several times, and the average execution time should be calculated.

Preprocessing time must be recorded separately from lookup time.

9.4 Filled Theoretical Comparison Table

The following table estimates the comparison counts for an **unsuccessful search**. Linear Search checks all records. Binary Search uses an estimated number of halving steps. Hash Search assumes a well-sized table and good key distribution.

Dataset Size	Linear Comparisons	Binary Comparisons (Approx.)	Hash Comparisons (Illustrative Average)
1,000	1,000	10	1–3
10,000	10,000	14	1–3
100,000	100,000	17	1–3
1,000,000	1,000,000	20	1–3

Explanation: The Linear Search values represent the worst case for an absent target. Binary Search values are approximate because the exact number depends on the array size and implementation. Hash comparison counts are illustrative assumptions, not guaranteed results; collisions may increase them.

9.5 Illustrative Execution-Time Table

The table below contains **hypothetical example values only** to demonstrate how results can be presented. They are not measured timings and must not be described as actual experimental results.

Dataset Size	Linear Search (ms)	Binary Search (ms)	Hash Lookup (ms)
1,000	0.010	0.002	0.002
10,000	0.100	0.003	0.002
100,000	1.000	0.004	0.002
1,000,000	10.000	0.005	0.002

These numbers illustrate the expected trend, not actual computer performance. Real timings depend on the implementation, hardware, compiler, input distribution and timer precision.

9.6 Expected Behaviour

Linear Search can require n comparisons when the target is absent.

Binary Search grows more slowly because it repeatedly halves the search range.

Hash-Based Search should normally perform a small number of comparisons when the table is appropriately sized and the hash function distributes keys well.

However, collisions may increase the number of comparisons.

9.7 Measurement Notes

Actual execution times should be collected using a high-resolution timer and repeated trials.

The computer, compiler, dataset and measurement method should be recorded. Preprocessing time should be measured separately from lookup time.

Printing output inside the timed search loop should be avoided because output operations can distort the measurements.

9.8 Conclusion

The theoretical comparison counts show that Binary Search scales better than Linear Search for large sorted datasets. Hash-Based Search is expected to provide fast exact-key lookup when collisions are controlled.

The illustrative timing table demonstrates the expected trend, but actual experimental conclusions must be based on measurements from the program.

TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

10.1 Theoretical Expectations

Linear Search has O(n) worst-case time because it may examine every record. Binary Search has O(log n) lookup time because it halves a sorted search interval. Hash-Based Search has expected O(1) lookup time when the table is appropriately sized and the hash function distributes keys effectively.

10.2 Comparison Table

Algorithm	Theoretical Expected Behaviour	Behaviour as Input Grows
Linear Search	O(n) worst case	Comparisons increase approximately in proportion to n
Binary Search	O(log n) worst case	Only a small number of extra comparisons are needed as n grows
Hash-Based Search	Expected O(1) lookup	Average lookup work should remain small if hashing is effective
Hash-Based Search — Worst Case	O(n)	Work may increase significantly if many keys collide

10.3 Linear Search

When the number of records doubles, the worst-case number of comparisons may also double.

For example, an unsuccessful search among 10,000 records can require more comparisons than a search among 5,000 records. This supports the linear growth expected from O(n).

10.4 Binary Search

Binary Search does not examine every record. It repeatedly eliminates approximately half of the remaining candidates. Doubling the dataset usually adds about one extra halving step. This supports the logarithmic growth expected from O(log n).

10.5 Hash-Based Search

With a suitable hash table, the number of comparisons for an average lookup should remain relatively small as the dataset grows.

However, collisions, table capacity and the hash function can affect measured performance.

10.6 Reasons for Differences Between Theory and Practice

Actual running times can be influenced by:

- Processor speed.
- Compiler optimisation.
- Memory access.
- Operating-system activity.
- Input distribution.
- Hash collisions.
- Timer precision.
- Preprocessing costs.

Therefore, measured times may not match theoretical predictions perfectly.

10.7 Preprocessing Costs

Binary Search requires sorted data. If the data is unsorted, sorting may take O(n log n).

Hash-Based Search requires building the hash table, which takes expected O(n).

For only one search, preprocessing may form a significant part of the total cost. For thousands of searches, the setup cost can be distributed across many lookups.

10.8 Conclusion

Theoretical analysis and experimental measurements complement each other. Complexity analysis predicts how algorithms scale, while testing shows how the implementation behaves under real conditions.

Any major difference should be investigated by checking the code, input data, measurement method and preprocessing costs.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

11.1 Recommendation

For a student-record system that performs frequent exact searches using registration numbers, Hash-Based Search is recommended as the primary lookup method.

Registration numbers act as unique keys, and the system usually needs to locate one particular student rather than search through a range of records.

11.2 Reason 1: Expected Lookup Speed

A well-designed hash table provides expected O(1) lookup time.

The algorithm calculates the location of a registration number and checks the corresponding slot or bucket.

This is useful when students, faculty members or administrators repeatedly request individual records.

11.3 Reason 2: Daily Insertions

New student records can be inserted into a hash table without sorting the entire collection after every insertion.

The system should monitor its load factor and increase the table capacity when necessary.

Collision handling must be implemented correctly to prevent records from becoming inaccessible.

11.4 Reason 3: Large Collections

For a collection containing many records, Linear Search may perform a large number of comparisons. Binary Search reduces the number of comparisons, but it requires sorted data.

Hash-Based Search is generally more suitable for frequent exact-match searches because its expected lookup time is constant.

11.5 Limitations of Hashing

Hash-Based Search has some disadvantages:

- It requires additional memory.
- It does not automatically maintain sorted order.
- Collisions can increase search time.
- Its worst-case lookup time is O(n).
- The implementation must handle duplicate keys and resizing correctly.

Therefore, a suitable hash function and table size are important.

11.6 When Binary Search Is Better

If the system frequently generates reports in registration-number order or performs range queries, sorted data is useful.

Binary Search offers O(log n) lookup time on sorted data. However, inserting into a sorted array may require shifting elements. For frequently updated systems, a database index or another suitable data structure may be more practical.

11.7 Why Not Only Linear Search?

Linear Search is easy to implement and works with unsorted data. However, its O(n) worst-case cost becomes expensive when the collection is large and many searches are performed.

11.8 Final Decision

Hash-Based Search is recommended for the primary exact-match lookup because it provides expected constant-time searching and supports incremental insertions. The final design should be validated through actual performance measurements, memory usage and collision testing.

If ordered reports and range searches are also important, a sorted structure or database index can be maintained alongside the hash-based lookup.

TASK 12 – RE-EVALUATE FOR 50,000 SEARCHES PER HOUR

12.1 Given Requirements

The student-record system has the following requirements:

- Number of records: 1,000,000.
- Searches per hour: 50,000.
- New records: inserted daily.
- Main operation: searching for a specific registration number.

Calculate the average search rate:

Searches per second = 50,000 / 3,600

Searches per second ≈ 13.89

Therefore, the system receives approximately 14 searches per second on average. The design must support fast lookups, daily updates, correctness and future growth.

12.2 Selected Data Structure

A hash table keyed by the unique registration number is recommended for the primary lookup operation. With an appropriate hash function and table size, the expected lookup time is O(1).

Building a hash table for n records requires expected O(n) time. Therefore, rebuilding the entire table for every new record should be avoided. New records should be inserted incrementally as part of the daily update process.

12.3 Collision Handling

A collision occurs when two different registration numbers map to the same hash-table location. The system can use separate chaining or open addressing to handle collisions.

The load factor is calculated as:

Load Factor = Number of Stored Keys / Number of Table Slots

If the load factor becomes too high, the system should increase the table capacity and redistribute the keys.

This helps reduce collisions and maintain efficient lookup performance.

12.4 Daily Record Insertion

Daily insertions should be handled carefully.

1. Validate the registration number.
2. Check whether the registration number already exists.
3. Store the new student record.
4. Add the registration number to the hash index.
5. Confirm that the record and index are consistent.
6. Record the update so that failures can be detected and recovered.

Duplicate registration numbers should be rejected or handled according to college policy.

12.5 Performance Testing

The system should be tested with one million records and realistic search requests.

The experiment should measure:

- Average lookup time.
- High-percentile lookup latency.
- Number of searches completed per second.
- Collision count.
- Memory consumption.
- Daily insertion time.
- Performance under concurrent requests.

Both successful and unsuccessful searches should be tested.

The system should also be tested while records are being inserted.

12.6 Reliability and Security

The implementation should validate user input and restrict access to authorised users. The system should maintain backups and logs so that failed updates can be recovered. If a database already stores the student records, a database index may be preferable to a manually maintained in-memory hash table because the database can provide persistence and concurrency management.

12.7 Limitations

Expected O(1) lookup is not a guarantee for every possible input. Poor hash distribution, excessive collisions or a high load factor can increase search time. Therefore, the hash function, table capacity and collision-handling method must be tested carefully. A sorted index may also be useful if the system needs frequent ordered reports or range queries.

12.8 Final Recommendation

For one million student records and 50,000 searches per hour, Hash-Based Search is a suitable primary design for exact registration-number lookups. The system should use incremental daily insertions, collision handling, load-factor monitoring and controlled resizing.

Finally, the design should be tested under realistic workloads before deployment. The decision should be supported by measured lookup latency, throughput, memory usage and insertion performance rather than theoretical complexity alone.

OVERALL CONCLUSION

This assignment compared Linear Search, Recursive Binary Search and Hash-Based Search for locating student records. Linear Search is simple and supports unsorted data, but its worst-case time complexity is O(n). Binary Search provides O(log n) lookup when the data is sorted. Hash-Based Search provides expected O(1) lookup when the hash function and table capacity are appropriate, although its worst-case time is O(n). For a system containing one million student records and receiving 50,000 searches per hour, Hash-Based Search is a suitable primary method for exact registration-number lookup. Daily insertions, collision handling, resizing, memory usage and reliability must be considered. The final design should be verified through actual experiments. Theoretical complexity explains expected growth, while measured results help confirm whether the implementation meets the system requirements.